DevOps and Automations TASK – 6

1.Write a shell script named deploy_playlist_app.sh that automates the deployment of a simple Node.js 'Music Playlist' API to your chosen cloud provider (AWS, Azure, or GCP). The script should install dependencies, start the app, and output the public URL.

Ans :

- Create the script
- nano deploy_playlist_app.sh
- Paste:
- #!/bin/bash

- set -e

- # AWS EC2 instance details
- EC2_USER="ec2-user"
- EC2_IP="YOUR_EC2_PUBLIC_IP"
- APP_DIR="/home/ec2-user/music-playlist-app"

- echo "Deploying Music Playlist API to $EC2_IP..."

- # Copy application files to EC2
- scp -r ./app/* "$EC2_USER@$EC2_IP:$APP_DIR/"

- # Connect to EC2 and deploy
- ssh "$EC2_USER@$EC2_IP" << 'EOF'

- set -e

- cd /home/ec2-user/music-playlist-app

- echo "Installing Node.js dependencies..."
- npm install

- echo "Stopping previous application..."
- pkill -f "node app.js" || true

- echo "Starting Music Playlist API..."
- nohup npm start > playlist.log 2>&1 &

- echo "Application started."
- EOF

- echo ""
- echo "Deployment completed successfully!"
- echo "Public URL: http://$EC2_IP:3000"
- Make it executable
- chmod +x deploy_playlist_app.sh
- Change the EC2 IP
- Replace:
- EC2_IP="YOUR_EC2_PUBLIC_IP"
- with your actual EC2 public IP:
- EC2_IP="13.234.XX.XX"
- Run the deployment

- ./deploy_playlist_app.sh
- The script will:
- Copy application
- ↓
- SSH into EC2
- ↓
- npm install
- ↓
- Stop old app
- ↓
- Start Node.js app
- ↓
- Print public URL
- Expected output
- Deploying Music Playlist API to 13.234.XX.XX...
- Installing Node.js dependencies...
- Stopping previous application...
- Starting Music Playlist API...
- Application started.

- Deployment completed successfully!
- Public URL: http://13.234.XX.XX:3000

**2.Create a YAML configuration file for GitHub Actions that automatically builds and deploys a 'Food Delivery Tracker' (a basic Express.js app) to Heroku whenever you push to the main branch.

Hint: Use the official Heroku GitHub Action and set up the required secrets for authentication.**

Ans :

- deploy.yml
- name: Deploy Food Delivery Tracker

- on:
- push:
- branches:
  - main

- jobs:
- build-and-deploy:
- runs-on: ubuntu-latest

- steps:
  - name: Checkout code
- uses: actions/checkout@v4

  - name: Install dependencies
- run: npm ci

  - name: Test application
- run: npm test --if-present

  - name: Deploy to Heroku
- uses: akhileshns/heroku-deploy@v3.14.15
- with:
  - heroku_api_key: ${{ secrets.HEROKU_API_KEY }}
  - heroku_app_name: ${{ secrets.HEROKU_APP_NAME }}

- heroku_email: ${{ secrets.HEROKU_EMAIL }}

- The Heroku deployment action supports HEROKU_API_KEY, app name, and email through its inputs.
- Your Express app
- Make sure your project has a package.json with a start script:
- {
- "name": "food-delivery-tracker",
- "version": "1.0.0",
- "main": "app.js",
- "scripts": {
- "start": "node app.js"
- },
- "dependencies": {
- "express": "^5.1.0"
- }
- }
- And app.js:
- const express = require("express");

- const app = express();
- const PORT = process.env.PORT || 3000;

- app.get("/", (req, res) => {
- res.send("Food Delivery Tracker");
- });

- app.listen(PORT, () => {
- console.log(`Food Delivery Tracker running on port ${PORT}`);
- });

- Add the Heroku secrets
- In GitHub:
- Repository → Settings → Secrets and variables → Actions → New repository secret
- Add:
- HEROKU_API_KEY
- HEROKU_APP_NAME
- HEROKU_EMAIL
- For example:
- HEROKU_APP_NAME = my-food-delivery-tracker
- Do not put the actual API key directly in the YAML file.
- Add a Procfile
- Create a file named:
- Procfile
- with:
- web: npm start
- The Heroku action documentation supports using a Procfile for the deployment.
- Commit and push
- git add .
- git commit -m "Add Heroku deployment workflow"
- git push origin main
- Check deployment
- Go to your GitHub repository → Actions → Deploy Food Delivery Tracker.
- The workflow will run:
- Push to main
- ↓
- Checkout code
- ↓
- npm ci

- ↓
- Run tests
- ↓
- Deploy to Heroku

3.Modify a given deployment script for a 'Movie Review' app so that it also runs database migrations before starting the server. Test your updated script by deploying the app to a free-tier cloud instance and confirming the migration runs.

Ans :

- Create New → Web Service.
- Connect your GitHub repository.
- Select the main branch.
- Choose Node.
- Set Build Command:
- npm install
- Set Start Command:
- npm start
- For the migration, preferably set Render's Pre-Deploy Command to:
- npm run migrate
- Choose the Free instance where available.
- Add any required database environment variables, such as:
- DATABASE_URL

4.Set up environment variable management for a 'Flipkart Wishlist' microservice by configuring your deployment automation

**to securely inject API keys and database credentials from a .env file or the cloud provider's secret manager.

Hint: Do not hardcode sensitive information in your scripts or code.**

Ans :

I configured secure environment-variable management for the Flipkart Wishlist microservice. API keys and database credentials are stored in a local .env file for development and in the cloud provider's environment-variable/secret manager for production. The .env file is excluded using .gitignore, while .env.example documents the required variables without exposing their values. The deployment script checks that required variables exist before starting the application.

5.Use ChatGPT or GitHub Copilot to generate a deployment automation script for a 'BookMyShow Event Reminder' app (can be any simple web app you have). Review the generated script, test it, and document any changes you had to make for it to work in your environment.

Ans :

Changes made**:** ChatGPT generated the initial deployment script. I reviewed it and changed npm install to npm ci for reproducible dependency installation. I also added set -e so the deployment stops when a command fails and used npm test --if-present because the placeholder application does not currently contain a test script. I tested the application locally and then configured GitHub Actions to deploy it automatically to an AWS EC2 instance whenever changes are pushed to main.